

* Unit V: Memory management [14M]

• Page replacement Algorithms:

① FIFO

- FIFO stands for First In First Out.
- This is the simplest page replacement algorithm. In this algorithm, operating system keeps track of all pages in the memory in a queue, oldest page is in the front of queue.
- When a page needs to be replaced page in front of the queue is selected for removal.

* Example:

- ① Consider following reference string
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1,
7, 0, 1

The 3 frames are initially empty.

7 0 1 2 0 3 0 4 2 3

7	7	7	2	2	2	4	4	4
	0	0	0	3	3	3	2	2
		1	1	1	0	0	0	3

Page no.: _____ Date: ____/____/____

0	3	2	1	2	0	1	0	1
0	7		0	0			7	0
0	7		1	1			1	7
2	7		3	2			2	0
3	7							1

Page fault = 15
 Page hit = 5

② Reference string - 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
 Frame size = 4

7 0 1 2 0 3 0 4 2

7	7	7	7	3	3
	0	0	0	0	4
		1	1	1	1
			2	2	2

3 0 3 2 1 2 0 1 7

3	3	2	2
4	4	4	7
0	0	0	0
0	1	1	1

0 1

Page fault = 10. Page hit = 10

② Optimal

- In this algorithm, pages are replaced which would not be used for longest duration of time in future.
- It has the lowest page fault rate of all algorithms.
- This algorithm uses a 'stack-based' algorithm for replacing pages. Using this, Belady's Anomaly can be removed.

* Example

① Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

7 0 1 2 0 3 0 4 2 3

7	7	7	2		2		2
	0	0	0		0		4
		1	1		3		3

0 3 2 1 2 0 1 7 0 1

2		2		7
0		0		0
3		1		1

Page fault = 9
Page hit = 11

③ LRU stands for Least Recently Used
- LRU replacement algorithm

- ③ LRU stands for Least Recently Used page replacement algorithm. This algorithm replacement policy replaces the page that has not been used for longest period of time.

- LRU replacement associates with each page the time of that page's last use.

- An LRU page-replacement may require substantial hardware assistance.

① Counters: In simplest case, we associate with each page-table entry a time-of-use field and add to CPU a logical clock or counter. The clock is incremented for every memory reference.

ii) Stack - Another approach to implementing LRU replacement is to keep a stack of page numbers. Whenever a page is referenced, it is removed from the stack and put on the top.

* Example.

① Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 2, 1, 2, 0, 1, 7, 0, 1

7	0	1	2	0	3	0	4	2
7	7	7	2	2		4	4	
	0	0	0	0		0	0	
		1	1	3		3	23	

3 0 3 2 1 2 0 1 7

4	0	1	1	1
3	3	3	3	7
2	2	2	0	0

0 1

Page fault = 12
Page hit = 8

Q. Reference string: 5, 6, 7, 8, 9, 7, 8, 5, 9, 7, 8, 7, 9, 6, 5, 6
(S-22)

(i) FIFO

5 6 7 8 9 7 8 5 9 7

5	5	5	8	8			8		7
	6	6	6	9			9		9
		7	7	7			5		5

8 7 8 7 9 6 5 6

7			7	6	6
8			8	8	5
5			9	9	9

Page fault = 11
Page hit = 5

⑧ Optimal

5 6 7 8 9 7 8 5

5	5	5	5	9			9
	6	6	8	8			5
		7	7	7			7

9 7 8 7 9 6 5 6

9		9	9
8		6	6
7		7	5

⑧ LRU

5 6 7 8 9 7 8 5

5	5	5	8	8			8
	6	6	6	9			6
		7	7	7			7

9 7 8 7 9 6 5 6

8	7	7			7	5
5	6	8			6	6
9	9	9			9	9

Page fault = 11
Page hit = 5

Basic memory management

Page no.:
Date: / /

- Two major memory management schemes are possible. Each approach divides memory into number of regions or partitions. Partitioning is a technique that divides a memory into multiple partitions. These partitions can be of different or same size.

- Types:

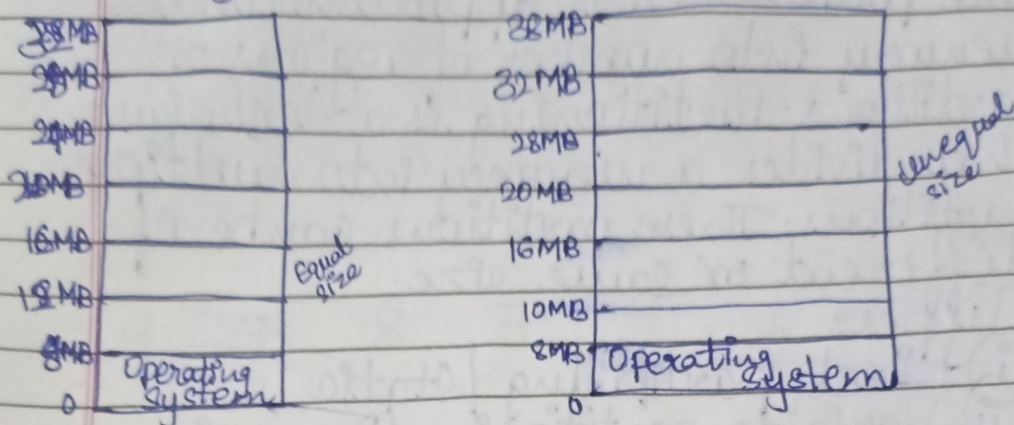
- i) Fixed partitioning / Static
- ii) Variable partitioning / Dynamic.

i) Fixed Size Memory partitioning:

- a) Memory is divided into number of fixed size partitions, which is called as fixed memory partitioning.
- b) Each partition contains exactly one process. The number of programs to be executed depends on number of partitions.
- c) When the partition is free, a selected process from input queue is loaded into free partition.
- d) When the process terminates, the partition becomes available for another process.
- e) The operating system keeps a table indicating parts of memory which are available & which are occupied.
- f) Initially, all memory is available for user processes and it is considered as one large block of available memory, a hole.

g) When a process arrives, large enough hole of memory is allocated to process.

h) Diagram:



i) Advantages:

- Simple to implement
- It requires minimal operating system software and processing overhead as partitions are fixed at time of system generation.

j) Disadvantages:

- Memory wastage
- Inefficient use of memory due to internal fragmentation.

ii) Variable partitioning

- a) When a process enters in main memory, it is allocated exact size that is required by that process.
- b) So in this method, partitions can vary in size depending on memory space required by a process entering in main memory.

- c) Operating system maintains a table indicating which parts of memory are available and which are occupied.
- d) When new process arrives and it needs space, system searches for available memory space in main memory.
- e) If it is available, then memory is allocated to process by creating a partition in memory.
- f) Example:
Consider following table with process & memory space.

Process	Memory space
P1	20M
P2	14M
P3	18M

operating system	8M
	56M

operating system	8M
Process 1	20M
	36M

operating system	8M
Process 1	20M
Process 2	14M

operating system	8M
Process 1	20M
Process 2	14M
Process 3	18M
	4M

- a) Advantages:
- i) No internal fragmentation
 - ii) More efficient use of memory
- b) Disadvantages:
- i) It suffers from external fragmentation
 - ii) It needs compaction.

▪ Free Space management techniques

A file system is responsible to allocate the free blocks to the file therefore it has to keep track of all free blocks present in the disk. There are mainly 4 approaches by using which, the free blocks in disk are managed: Bit vector, Linked List, Grouping, Counting.

i) Bit vector:

- i) The free-space list is implemented as bit map or bit vector.
- ii) Each block is represented by 1 bit. If block is free, the bit is 1; if block is allocated, the bit is 0.
- iii) Consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13 are free and rest of the blocks are allocated. The free space bitmap would be:

001111001111100

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	1	1	1	0	0	1	1	1	1	1	1	0	0

0 = Allocated block

1 = Free block

(i) The main advantage of this approach is its relative simplicity & its efficiency in finding first free block or 'N' consecutive free blocks on disk.

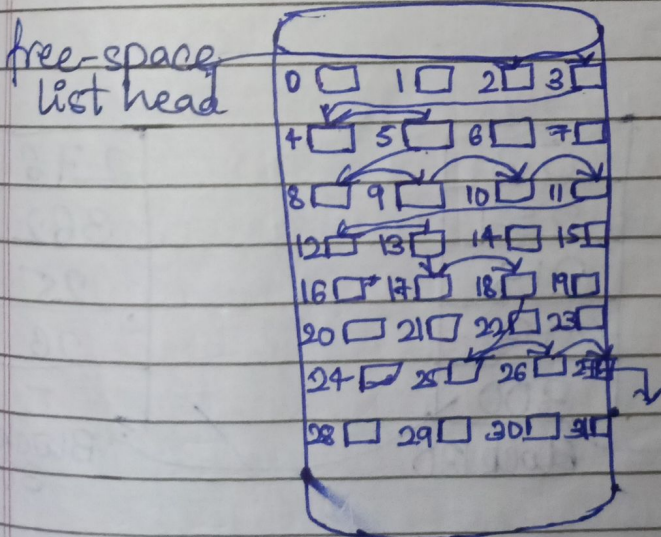
(ii) Linked List:

In this approach, the free disk blocks are linked together i.e. a free block contains a pointer to next free block.

(iii) The block number of very first disk block is stored at a separate location on disk and is also cached in memory.

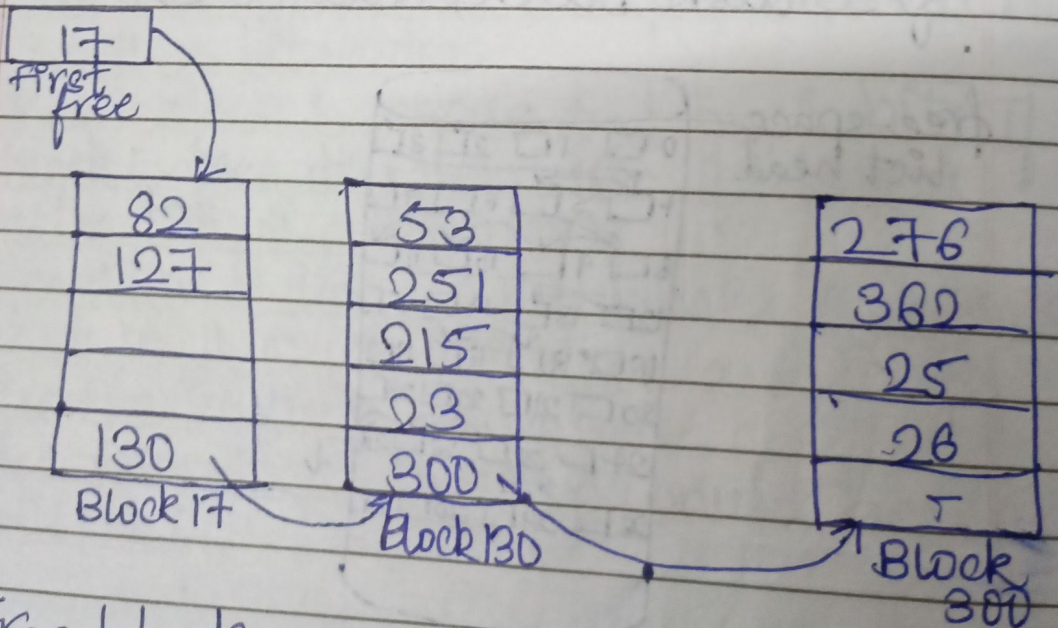
In this approach, link all disk blocks together, keeping a pointer to the first free block.

(iv) This block contains a pointer to next free disk block, and so on.



⑤ Grouping

- i) It is a free space management technique for a modification of free list method.
- ii) In grouping, there is a modification of this approach would store addresses of 'n' free blocks in first free block.
- iii) The first n-1 of these is actually free. The last one is disk address of another block containing addresses of another 'n' free block.
- iv) The importance of this implementation is that the addresses of a large number of free blocks can be found quickly.
- v) In which a disk block contains addresses of many free blocks & block containing free block pointers will get free when those blocks are used.



Free blocks are:

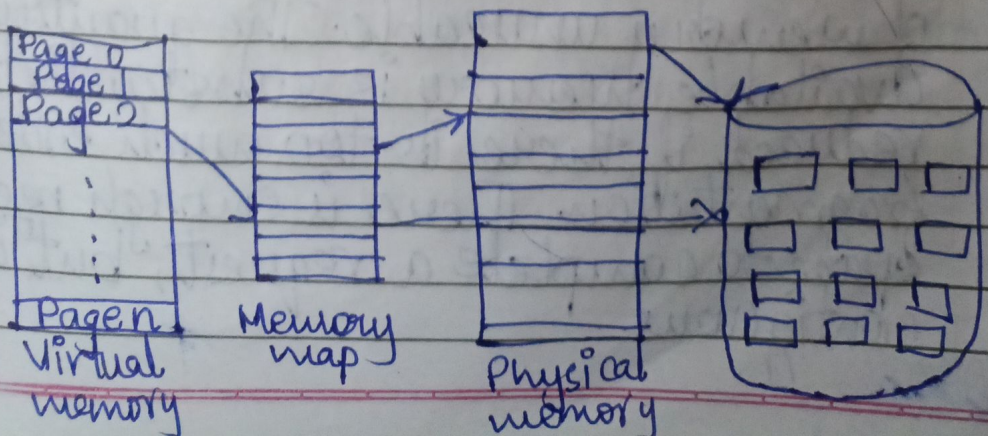
82, 127, 53, 251, 215, 23, 276, 362, 25, 26

demand page - It is a method of virtual memory management. With demand page, virtual memory are only loaded, when they are demanded during program execution.

Date: / /

• Virtual Memory

- i) It is the separation of user logical memory from physical memory.
- ii) This separation allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available.
- iii) Virtual memory makes the task of programming much easier, because the programmer no longer needs to worry about amount of physical memory available, or about what code can be placed in overlays, but can concentrate instead on problem to be programmed.
- iv) It allows execution of processes that may not be completely in memory.
- v) The basic idea behind virtual memory is that combined size of program, data & stack may exceed the amount of physical memory available for it.
- vi) The OS keeps those parts of program currently in use in main memory & rest on the disk.



▪ Fragmentation

- Fragmentation refers to an unwanted problem that occurs in OS in which process is unloaded and loaded from memory, and free memory space is fragmented. These ^{areas} fragmented areas ^{are} into small pieces of memory that cannot be allocated to incoming processes. It is called fragmentation.

Types:

① Internal Fragmentation:

When a process is allocated to memory block and if the process is smaller than amount of memory requested, a free space is created in the given memory block. Due to this, the free space of memory block is unused, which causes internal fragmentation.

② External Fragmentation:

It happens when a dynamic memory allocation method allocates some memory but leaves a small amount of memory unusable. The quantity of available memory is substantially reduced if there is too much external fragmentation. There is enough memory space to complete a request, but it is not contiguous.

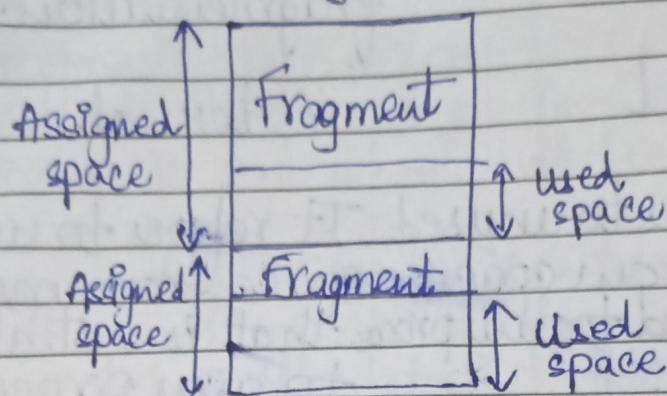
- Difference: Internal & External fragmentation.

Internal

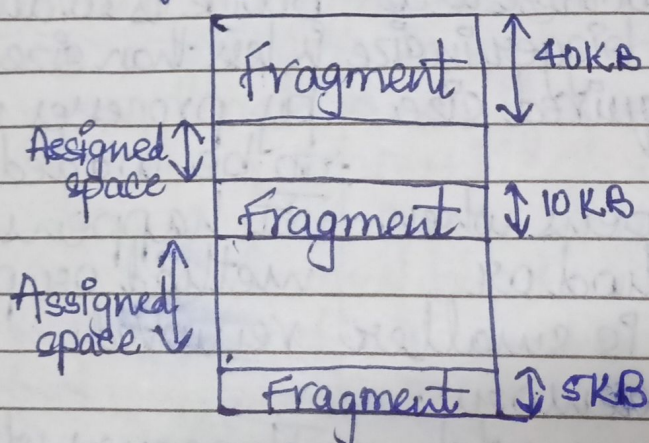
External

- | | |
|--|---|
| <p>(i) It refers to unused space from space allocated to any process.</p> <p>(ii) Internal fragmentation occurs when memory block allocated to a process is bigger in size than required size.</p> <p>(iii) It happens when the method or process is smaller than memory.</p> <p>(iv) It occurs when memory is divided into fixed-size partitions & with paging.</p> <p>(v) It occurs in worst fit memory allocation method.</p> | <p>It refers to unused space from memory that is not allocated to any process.</p> <p>External fragmentation occurs when free memory block is available but is less than size required by processes waiting to be loaded in memory.</p> <p>It happens when method or process is removed.</p> <p>It occurs when memory is divided into variable size partitions based on size of processes & with segmentation.</p> <p>It occurs in best fit & first fit memory allocation method.</p> |
|--|---|

Internal fragmentation diagrams



External fragmentation diagram:



Memory allocation methods:

- ① First fit - Allocate the first hole (or free block) that is big enough for the new process. Searching can start either at beginning of the set of holes or where the previous first fit search ended. We can stop searching as soon as we find a large enough free hole. First fit is generally faster.

② Best fit - Allocate smallest hole that is big enough. We search the entire list, unless the list is kept ordered by size. This strategy produces smallest left over hole.

③ Worst fit - Allocate the largest hole. Again we must search the entire list, unless it is sorted by size.

Q. Consider following memory map & assume a new process P4 comes with requirements of 6KB. Locate this process in memory using first fit, best fit & worst fit.

OS
P1
<Free> 12KB
P2
<Free> 19KB
P3
<Free> 7KB

Solution:

① first fit -

OS
P1
P4 6KB
<Free> 6KB
P2
<Free> 19KB
P3
<Free> 7KB

ii) Best fit -

OS
P1
<Free> 10 KB
P2
<Free> 19 KB
P3
P4 6 KB
<Free> 2 KB

iii) Worst fit -

OS
P1
<Free> 12 KB
P2
P4 6 KB
<Free> 13 KB
P3
<Free> 7 KB

Q. Solve using page replacement algorithms:
 12, 3, 4, 5, 1, 2, 5, 1, 2, 3, 4, 5, 1, 6, 7,
 8, 7, 8, 9, 7, 8, 9, 5, 4, 4, 5, 4, 2
 Page frame = 4

i) FIFO

12 3 4 5 1 2 5 1 2

12	12	12	12	1	1	
	3	3	3	3	2	
		4	4	4	4	
			5	5	5	

3 4 5 1 6 7 8 7

1	1	5	5	5	5	8
2	2	2	1	1	1	1
3	3	3	3	6	6	6
5	4	4	4	4	7	7

8 9 7 8 9 5 4 4

8
9
6
7

8	8
9	9
5	5
7	4

5 4 2

9
9
5
4

Page fault = 17
 Page hit = 11

11 Optimal

12 3 4 5 1 2 5 1

12	12	12	12	1	1	1	
	3	3	3	3	3	3	
		4	4	4	4	5	
			5	5	2	2	

2 3 4 5 1 6 7 8

		1			6	7	7
		4			4	4	4
		5			5	5	5
		2			2	2	8
7	8	9	7	8	9	5	4
				x	x	x	

7
9
5
8

1
4
5
8

4
2
5
8

4 5 4 2

100
111

LRLl

12 3 8 4 5 1 2 5 1

12	12	12	12	1	1
	3	3	3	3	2
		4	4	4	4
			5	5	5

2 3 4 5 1 6 7 8

1	1	5	5	5	5	8
2	2	2	1	1	1	1
3	3	3	3	6	6	6
5	4	4	4	4	7	7

7 8 9 7 8 9 5 4

8
9
6
7

8
9
5
7

8
9
5
4

4 5 4 2

2
9
5
4

When processor needs to execute a page & if that page is not available in ^{main} memory then this situation is called page fault.

• Page fault

A page fault in OS is a computational memory error that occurs when a program running in CPU attempts to access a page of data from its address space.

• Paging

- i) Paging is a memory management scheme that eliminates need for a contiguous allocation of physical memory.
- ii) The process of retrieving processes in form of pages from secondary storage into main memory is known as paging.
- iii) The basic purpose of paging is to separate each procedure into pages.
- iv) Additionally, frames will be used to split the main memory.

• Segmentation

- i) A process is divided into segments. The chunks that a program is divided into which are not necessarily all of exact sizes are called segments.
- ii) Segmentation gives users view of the process which paging does not provide.

• Paging & Segmentation

Paging

- i In Paging, we break a process address space into blocks known as pages.
- ii The pages are blocks of fixed size.
- iii This technique is comparatively much faster in accessing memory.
- iv The available memory determines the individual page sizes.
- v The paging techniques may underutilize some of the pages - thus leading to internal fragmentation.
- vi In case of paging, page table leads to storage of page data.

Segmentation

In case of segmentation, we break a process address space into blocks known as sections/segments.

The sections/segments are blocks of varying sizes.

This technique is comparatively much slower in accessing memory.

The user determines the individual segment sizes.

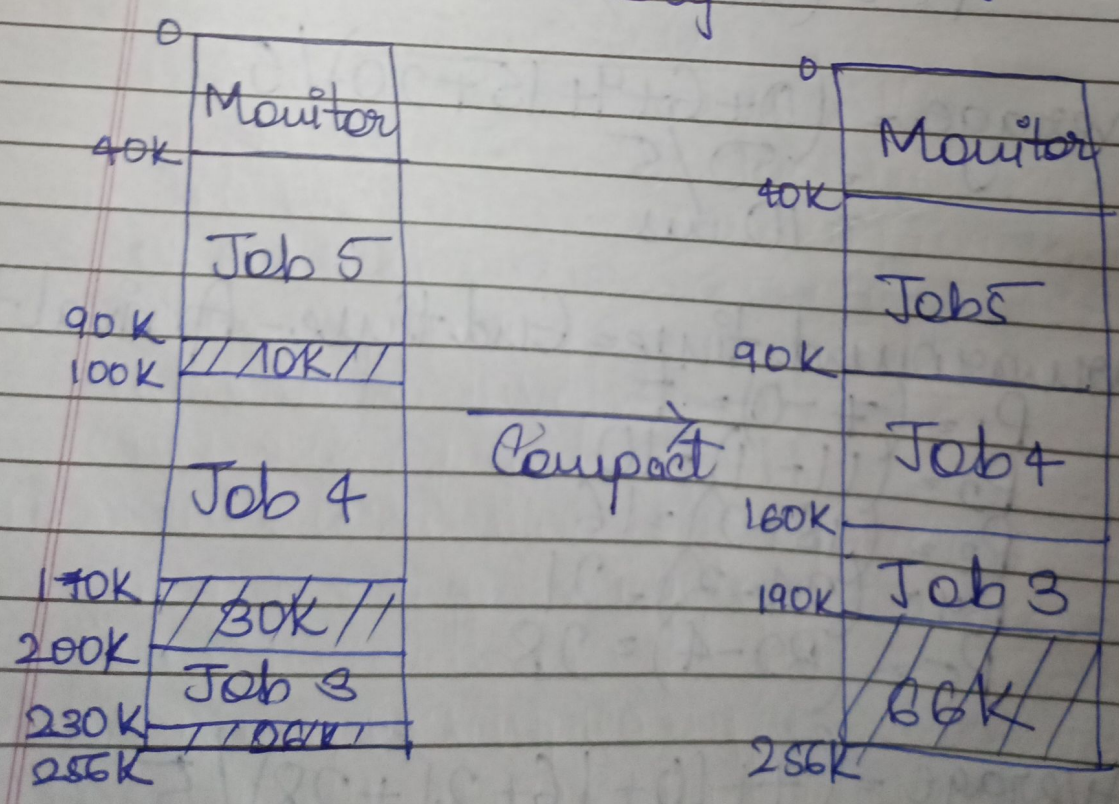
The segmentation technique may not use some of the memory blocks at all. Thus, it may lead to external fragmentation.

In case of segmentation, the segmentation table leads to storage of segmentation data.

- Demand page
- It is a memory management technique used in virtual memory systems where pages enter main memory only when requested or needed by CPU.
- The operating system loads only necessary pages of program into ~~runtime~~, instead of loading entire program into memory at start.
- Page hit
- When CPU attempts to obtain needed page from main memory & page exists in main memory it is referred as page hit.

• Swapping-Compaction

- i) Compaction is a method used to overcome external fragmentation problem. All free blocks are brought together as one large block of free space.
- ii) The collection of free space from multiple non-contiguous blocks into one large free block in a system's memory is called compaction.
- iii) Compaction is possible only if relocation is dynamic, at execution time using base & limit registers.
- iv) The simplest compaction algorithm is to simply move all jobs towards one end of memory, all holes move in other direction producing one large hole of available memory.



Safety algorithm

1. Let $Work$ and $Finish$ be vectors of length ' m ' and ' n ' respectively.

Initialize: $Work = Available$
 $Finish[i] = false$;

for $i = 1, 2, 3, 4 \dots, n$

2. Find an i such that both

a) $Finish[i] = false$

b) $Need_i \leq Work$

if no such i exists go to step 4

3. $Work = Work + Allocation[i]$

$Finish[i] = true$

go to step 2

4. if $Finish[i] = true$ for all i then system is in safe state.